

Optimizing Elevator Control: A Study of AI-Based Learning Methods

Tuna Gül* and Yiğit Efe Erdoğan† *Department of Civil Engineering, Boğaziçi University
Email: *tuna.gul@std.boun.edu.tr, †yigit.efe@nfrproducts.com

Abstract—This paper explores the optimization of elevator control systems through the comparison of traditional and artificial intelligence (AI)-based algorithms. Efficient elevator management is critical in modern high-rise buildings, directly impacting passenger satisfaction, energy consumption, and operational costs. We implemented two distinct control strategies within a Rust-based simulation environment: a *Simple Elevator Control Algorithm*, which employs deterministic dispatching methods, and a *Neuroevolution-Based Control Algorithm*, leveraging a neural network trained through evolutionary strategies to adaptively assign target floors to elevators.

The simulation models realistic elevator dynamics and passenger behaviors, incorporating key metrics such as average waiting time, energy consumption, system throughput, response time, and overall efficiency. Preliminary results indicate that while the Simple Elevator Control Algorithm offers predictable and low computational overhead performance, the Neuroevolution-Based approach holds potential for enhanced adaptability and efficiency in dynamic passenger flow scenarios. However, the training process for the neural network is ongoing, and comprehensive performance evaluations are pending.

This study lays the groundwork for developing more intelligent and responsive vertical transportation systems. Future work will focus on completing the training of the neural network, refining the control algorithms, and integrating additional AI-driven strategies to further enhance system performance. By advancing these methodologies, we aim to contribute to the creation of elevator systems that can dynamically respond to varying passenger demands, thereby optimizing both energy usage and passenger experience in urban environments.

Index Terms—Elevator control algorithms, Artificial intelligence, Neuroevolution, Simulation, Energy consumption, Waiting time.

I. INTRODUCTION

In the landscape of rapidly urbanizing cities, high-rise buildings have become a ubiquitous feature, necessitating efficient vertical transportation systems to facilitate the movement of occupants. Elevators are indispensable in such structures, directly influencing passenger satisfaction, energy consumption, and the overall operational efficiency of buildings. As the demand for more responsive and energy-efficient elevator systems grows, traditional control algorithms face increasing scrutiny for their limitations in handling dynamic and unpredictable passenger flows.

Traditional elevator control algorithms, such as the *Nearest Car Algorithm* and the *Simple Collective Control Algorithm*, have been widely adopted due to their straightforward implementation and reliable performance under consistent traffic conditions. The Nearest Car Algorithm assigns elevator

requests to the closest available car, minimizing immediate wait times, while the Simple Collective Control focuses on collective scheduling to optimize service across multiple floors. Despite their effectiveness in stable environments, these methods often struggle to adapt to fluctuating passenger demands, leading to increased waiting times and higher energy consumption during peak usage periods.

Advancements in artificial intelligence (AI) and machine learning have opened new avenues for optimizing elevator control systems. AI-based algorithms, particularly those utilizing evolutionary strategies and neural networks, offer the potential for adaptive learning and real-time decision-making. These methods can analyze complex patterns in passenger behavior, predict future demand, and dynamically adjust elevator assignments to optimize performance metrics such as wait times and energy usage.

This study aims to compare traditional and AI-based elevator control algorithms within a comprehensive simulation framework that models realistic elevator dynamics and passenger interactions. Specifically, we implemented two control strategies:

- **Simple Elevator Control Algorithm:** A deterministic approach that assigns target floors based on predefined dispatching rules, serving as a baseline for performance comparison.
- **Neuroevolution-Based Control Algorithm:** An AI-driven method utilizing a neural network with three hidden layers, trained through evolutionary strategies to adaptively assign target floors in response to dynamic passenger flows.

The simulation environment, developed in Rust, incorporates detailed models of elevator motion, motor dynamics, and passenger behaviors. Key performance metrics evaluated include average waiting time, energy consumption, system throughput, response time, and overall efficiency. These metrics provide a holistic view of each algorithm's effectiveness in managing elevator operations under varying traffic conditions.

By systematically comparing traditional and AI-based elevator control strategies, this study seeks to advance the development of intelligent vertical transportation systems capable of meeting the demands of modern urban environments.

II. METHODOLOGY

This section details the experimental setup and the algorithms employed in the study. The methodology encompasses the simulation environment, physical dynamics, external variables influencing the tests, human interaction through the graphical user interface (GUI), and the implementation of both traditional and artificial intelligence-based control algorithms.

A. Experimental Setup

1) Physical Dynamics Modeling

Accurate modeling of physical dynamics is paramount for simulating realistic elevator operations. The simulation integrates fundamental physical laws and mechanical principles to replicate the behavior of an actual elevator system. This subsection provides a comprehensive explanation of the mathematical formulations and implementation details used to model the elevator's motion and motor dynamics.

a) Elevator Motion Dynamics

The vertical movement of the elevator is governed by Newton's second law of motion, which relates the net force acting on the elevator to its acceleration. The primary forces considered in the simulation are gravitational force and the force exerted by the motor. The elevator's mass includes both the cabin mass (m_e) and the dynamic passenger load (m_p).

- **Gravitational Force (F_g):** The force due to gravity acting on the elevator is calculated as:

$$F_g = m_{\text{total}} \cdot g$$

where:

- $m_{\text{total}} = m_e + m_p$ is the total mass of the elevator system.
- $g = 9.81 \text{ m/s}^2$ is the acceleration due to gravity.
- **Motor Force (F_m):** The force exerted by the motor is determined by the motor torque (τ_m) and the radius of the output shaft (r_p):

$$F_m = \frac{\tau_m}{r_p}$$

- **Net Force (F_{net}):** The net force acting on the elevator is the difference between the motor force and the gravitational force:

$$F_{\text{net}} = F_m - F_g$$

- **Acceleration (a):** Applying Newton's second law, the acceleration of the elevator is:

$$a = \frac{F_{\text{net}}}{m_{\text{total}}} = \frac{F_m - F_g}{m_{\text{total}}}$$

b) Acceleration and Capacity Constraints

To ensure passenger comfort and safety, the elevator's acceleration is constrained based on ISO 2631-4:2001 standards. According to these guidelines, passengers may experience slight discomfort at acceleration levels between 0.315 m/s^2 and 0.63 m/s^2 . Therefore, an acceleration limit of $a_{\text{max}} = 0.6 \text{ m/s}^2$ is implemented, which is near the upper bound of acceptable discomfort. This value is configurable via the simulation interface to allow for parameter adjustments as needed.

$$-a_{\text{max}} \leq a \leq a_{\text{max}}$$

c) Simulation Parameters

Several key parameters define the physical characteristics of the elevator system. These parameters are configurable through the simulation interface, allowing for flexibility and experimentation.

• Elevator Configuration:

- **Cabin Size:** The elevator cabin is modeled with dimensions $3 \text{ m} \times 2 \text{ m}$ (width $\times$ depth), and the door width is set to 1 m. These specifications regulate the maximum number of occupants permitted within the cabin during operation.

• Capacity Limits:

- **Maximum Load:** The elevator has a maximum load capacity of 1000 kg, corresponding to approximately 13 passengers based on average weight distributions. While the simulation imposes no strict limit on the number of passengers, individual passenger classes include weight data generated through a normal distribution, aligned with real-world average weights and standard deviations. Additionally, a circular area representing the space occupied by each passenger constrains the total number of occupants. Detailed calculations and modeling for passenger weight distribution are provided in Section II-A2.

• Building Configuration:

- **Number of Elevators:** The simulation includes two elevators operating within the system.
- **Number of Floors:** There are seven floors in the building.
- **Floor Heights:** Floor-to-floor distances are standardized at 3 m, except between the 4th and 5th floors, where a 5 m height is used to simulate architectural variations.

d) Motor Dynamics

The elevator motor is modeled as a Permanent Magnet Synchronous Motor (PMSM), selected for its ability to deliver constant torque across a range of speeds, ensuring smooth and reliable elevator operations. The motor dynamics are meticulously simulated to reflect real-world behavior, focusing on torque regulation, speed control, and energy consumption.

Torque and RPM Relationship

In a PMSM, the torque (τ_m) remains relatively constant over varying speeds, which is advantageous for elevator applications where consistent force is necessary to move the cabin smoothly. The motor speed is constrained by four parameters: `softRPM`, `softCurrent`, `hardRPM`, and `hardCurrent`. The *soft limiter*, also referred to as *soft-cut parameters*, can be adjusted through the interface settings, allowing for user-defined limitations. These parameters ensure compliance with user-imposed constraints. Additionally, *hardRPM* and *hardCurrent* values are sourced from the motor's official documentation and are enforced to prevent speed and current limits from being exceeded.

The relationship between motor torque, angular velocity (ω), and linear speed (v) of the elevator is defined by:

$$\omega = \frac{v}{r_p}$$

where r_p is the radius of the output shaft connected to the pulley system.

Proportional-Integral-Derivative (PID) Control

A PID controller regulates the motor's current to maintain the desired elevator speed. The controller minimizes the error between the target speed (v_{target}) and the actual speed ($v(t)$) by adjusting the motor current ($I(t)$). The PID control law is expressed as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

where:

- $e(t) = v_{\text{target}}(t) - v(t)$ is the speed error.
- K_p , K_i , and K_d are the proportional, integral, and derivative gains, respectively.

The PID controller outputs a control signal ($u(t)$) that adjusts the motor current to correct the speed error. This ensures that the elevator responds accurately to speed commands while maintaining stability and minimizing overshoot.

Motor Sample Data Interpolation

Motor characteristics such as efficiency, voltage, and torque are derived from empirical data stored in `motor_samples.csv`. The simulation interpolates between data points to estimate motor properties at arbitrary current levels, enabling continuous and realistic motor behavior.

$$P_{\text{mech}} = \eta \cdot V \cdot I$$

where:

- P_{mech} is the mechanical power output.
- η is the motor efficiency.
- V is the motor voltage.
- I is the motor current.

This interpolation ensures that motor properties such as torque (τ_m) and RPM (ω) are accurately modeled based on the current input, as demonstrated in the `motor_samples.rs` implementation.

e) Motion Profiles

The elevator follows predefined motion profiles to ensure smooth operation. These profiles are designed to comply with acceleration and speed limits while optimizing travel time.

- **Constant Acceleration and Deceleration:** The elevator accelerates uniformly until it reaches the maximum allowed speed or until it needs to decelerate to stop at the target floor.
- **Cruising Speed:** The maximum speed (v_{max}) is determined based on the distance to be traveled and acceleration limits. The elevator may not reach v_{max} if the distance between floors is too short.

The distance required to accelerate to v_{max} is:

$$d_{\text{accel}} = \frac{v_{\text{max}}^2}{2a_{\text{max}}}$$

Similarly, the total travel distance (d) must satisfy:

$$d \geq 2d_{\text{accel}}$$

If this condition is not met, the elevator adjusts its v_{max} accordingly.

f) Energy Consumption Modeling

Energy consumption is tracked to evaluate the efficiency of control algorithms. The total energy used (E_{total}) is calculated by integrating the power consumption over time:

$$E_{\text{total}} = \int_{t_0}^{t_f} P(t) dt$$

where $P(t)$ is the instantaneous power consumption, computed as:

$$P(t) = V(t) \cdot I(t)$$

In the simulation, regenerative braking and advanced motor characteristics like back-EMF are not currently modeled but are identified as areas for future development. The motor data has been enriched through estimation methods to include parameters such as inductance and stator resistance, which can facilitate the incorporation of these features in subsequent iterations.

g) Implementation Details

The Rust code is divided into two files: `lib.rs` and `main.rs`. All system components are contained within `lib.rs`, while `main.rs` imports and runs them. Additionally, any structures in `lib.rs` intended for Python are momentarily accessible in `python_lib`. Key components include:

- **Time Integration:** The simulation employs a fixed time step (Δt) to update state variables in every loop. The system was restructured to operate independently of the computer's internal clock. This modification resolved issues caused by delays in updating PID loops when the computer operated slowly, ensuring the system now functions seamlessly.
- **Population Configuration:** A variable named `hour_length` was introduced in the population folder. To effectively test the simulation's boundaries, a sufficient number of entities was required; this variable allows us to adjust the length of an hour. By shortening an hour while keeping the entity count constant, we achieve a denser simulation. Consequently, the `time_multiplier` variable is no longer needed and has been removed from the system.

- **Elevator Class:** The `Elevator` class represents the structure and functionality of an elevator system, integrating physical properties and control mechanisms. Key attributes include:

- **Floors and Positioning:** The `floors` attribute stores floor heights, while `current_height` tracks the elevator's vertical position, updated in real-time as it moves.
- **Control System (PID):** A `PIDController` instance, `height_pid`, manages the elevator's height by adjusting speed to minimize overshoot and maintain accuracy when approaching target floors.
- **Mass and Load Management:** `elevator_mass` and `elevator_counter_mass` represent the elevator's mass and counterweight, respectively, which stabilize movement. `max_load` defines the maximum allowable weight, while `current_load` is dynamically updated based on onboard entities.
- **Motor and Energy Tracking:** The `ElevatorMotor` instance, `motor`, controls speed, acceleration, and energy usage, interfacing with the PID to apply precise force for smooth and safe movement.
- **Gravity and Force Calculations:** The `gravity` attribute is used to compute required forces during movement, ensuring accurate motor force application based on load and counterweight.
- **Entities and Capacity Management:** A vector, `entities`, stores all `Boardable` objects (such as people or items), allowing the elevator to manage load capacity and area. The `area` and `current_area` values ensure sufficient space before boarding.
- **Target Floor Management:** The `target_idx` attribute keeps track of the target floor index, guiding the elevator toward designated floors.
- **Debugging and Plotting:** The optional `line_plotter` attribute, if enabled, generates a height vs. time plot for debugging.

Key methods include:

- `update`: Adjusts position, speed, and energy usage in response to elapsed time. Calculates target speed with PID, updates motor settings, and movement data.
- `set_target`: Sets the target floor and adjusts PID settings, initiating movement towards the target.
- `can_board`: Ensures the elevator is stopped and aligned with a floor, allowing boarding if conditions are met.
- `load` and `unload`: Manage entity boarding and deboarding, adjusting load and area values as entities enter and exit.
- `calculate_target_speed`: Computes the target speed for smooth arrival at the destination floor.
- `calculate_motor_force`: Determines necessary force to reach desired acceleration, considering mass and gravitational effects.

- **Motor Class:** The `ElevatorMotor` class models the behavior of the elevator's motor system, implemented as a Permanent Magnet Synchronous Motor (PMSM) to ensure consistent torque delivery across varying speeds. Key attributes include:

- **Motor Samples and Properties:** The `motor_samples` vector stores predefined motor performance samples, allowing simulation of motor behavior at different current levels. `current_properties` represents the motor's real-time state.
- **Control System (PID):** The `speed_pid` controller dynamically adjusts the motor's output to reach target speeds efficiently, based on real-time feedback.
- **Gearbox Ratio and Force Limits:** The `gearbox_ratio` scales motor output to appropriate speed and torque ranges, while `max_force` defines the maximum allowable force output at the gearbox's shaft.
- **Current Management and Energy Tracking:** `current_current` holds the current applied to the motor, adjusted by PID. `total_energy_used` records cumulative energy consumption over time.
- **Debugging and Plotting:** The optional `line_plotter` enables visualization of the motor's RPM over time, providing insight into performance and response characteristics.

Key methods include:

- `from_file` and `new`: Initializes a motor instance using parameters from a file, including motor samples, PID settings, and force limits.
- `set_target_speed` and `get_current_speed`: Allows setting a desired speed and retrieves the motor's current speed, both in terms of gearbox output, facilitating consistent speed control.
- `update`: Updates the motor's state based on the elapsed time, adjusting current and calculating torque requirements to maintain target speed.
- `update_energy_used`: Tracks the energy consumed during each update cycle, allowing accurate energy monitoring.
- `plot`: If enabled, records RPM data for each update to the `line_plotter` for later analysis.
- **Configuration Files:** Parameters are loaded from YAML and CSV files, allowing easy adjustments without changing the codebase. The modular design in `lib.rs` connects to main functionality via `main.rs`, so configuration updates propagate smoothly across the simulation.
- **Energy Tracking:** The simulation tracks energy use based on motor power input over time, offering insights into control efficiency. Energy values in kilowatts (KW) were used, and total cost was calculated by multiplying by a unit price of 2.7, reflecting current electricity rates.

h) Mathematical Modeling Summary

The combined equations governing the elevator's motion and motor control are summarized as follows:

$$\omega = \frac{v}{r_p}$$

$$P_{\text{mech}} = \eta \cdot V \cdot I$$

$$F_m = \frac{\tau_m}{r_p}$$

$$F_{\text{net}} = F_m - m_{\text{total}} \cdot g$$

$$a = \frac{F_{\text{net}}}{m_{\text{total}}} = \frac{F_m - m_{\text{total}} \cdot g}{m_{\text{total}}}$$

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

These equations collectively ensure that the elevator responds accurately to speed commands while adhering to physical constraints and maintaining energy efficiency.

i) Future Enhancements

While the current simulation model provides a robust foundation for evaluating elevator control algorithms, several additional physical factors can be incorporated to enhance realism:

- **Regenerative Braking:** Implementing energy recovery mechanisms during deceleration phases to improve energy efficiency.
- **Frictional Forces:** Modeling friction between the elevator and guide rails, as well as internal cabin friction, to more accurately simulate real-world dynamics.
- **Wind Forces:** For high-speed elevators, air resistance can significantly impact performance and energy consumption.
- **Guide Rail Interaction:** Detailed modeling of interactions between the elevator cabin and guide rails to capture lateral forces and vibrations.

These enhancements are planned for future iterations to improve simulation accuracy and applicability. Motor data has been enriched with estimated parameters, such as inductance and stator resistance, to support the integration of back-EMF and regenerative braking, enabling more precise modeling in upcoming studies.

j) Conclusion

The simulation's detailed physical dynamics model both elevator motion and motor behavior, offering a realistic environment for testing various elevator control algorithms. With constant torque PMSM motor characteristics, precise acceleration limits, and configurable parameters, the simulation closely mirrors real-world elevator operations. Future enhancements will further increase model fidelity, supporting more comprehensive evaluations and optimizations.

2) External Setup Variables for Testing

To accurately evaluate the performance of different elevator control algorithms, it is essential to simulate realistic human traffic patterns within the elevator system. This involves modeling passenger arrival rates, demographic characteristics, boarding and alighting times, preferences for stairs versus elevators, and commuting activities. This subsection details the external variables incorporated into the simulation, supported by mathematical formulations and data derived from reputable sources.

a) Passenger Arrival Rates

Passenger arrival rates are critical for simulating demand on the elevator system. Arrival rates are based on real-world data from the Istanbul Metropolitan Municipality's (İBB) hourly rail system usage statistics [1]. This dataset provides average hourly passenger counts, which are adapted to represent elevator call frequencies in a high-traffic metro station context.

Assuming that passenger arrivals follow a Poisson distribution, the expected number of elevator calls per minute, $\lambda(t)$, during hour t is calculated as:

$$\lambda(t) = \frac{N_{\text{metro}}(t) \cdot P_{\text{elevator}}}{60}$$

where:

- $N_{\text{metro}}(t)$ is the average number of passengers entering the metro system during hour t .
- P_{elevator} is the probability that a passenger will choose to use the elevator over the stairs.

b) Stairs vs. Elevator Preferences

Passenger preferences for using stairs or elevators significantly influence elevator demand. These preferences are derived from the study by Kinsey et al. [2], which investigated human factors associated with elevator usage during evacuations using an online survey. Although the context is evacuation scenarios, the behavioral insights are applicable to daily elevator usage, especially in high-rise buildings or metro stations.

Key findings from the study include:

- **Base Case Preferences:** In normal conditions, 87.8% of participants would always or sometimes consider using the stairs to travel *down*, and 84.2% to travel *up*. Slightly more participants are willing to use the stairs when descending than ascending.
- **Effect of Queues:** When faced with a queue in the elevator waiting area, the willingness to use the stairs increases slightly. For descending, 89.4% would consider the stairs, and for ascending, 87.3% would consider the stairs.
- **Group Influence:** Traveling in a group decreases the likelihood of choosing stairs. Only 81.0% would consider stairs when descending, and 76.4% when ascending.
- **Average Floors Willing to Walk:** On average, participants are willing to walk 6.7 floors down and 4.7 floors up in the base case. This indicates that people are prepared to walk approximately 2.0 more floors when descending compared to ascending.

Using this data, the probability $P_{\text{stairs}}(f, d)$ that a passenger will choose the stairs over the elevator is modeled as a function of the number of floors f they need to travel and the direction d (up or down):

$$P_{\text{stairs}}(f, d) = \begin{cases} p_0(d) & \text{if } f \leq F_{\text{max}}(d) \\ 0 & \text{if } f > F_{\text{max}}(d) \end{cases}$$

where:

- $p_0(d)$ is the base probability of choosing the stairs in direction d .
- $F_{\text{max}}(d)$ is the maximum number of floors participants are willing to walk in direction d .

From the study:

- For descending ($d = \text{down}$):
 - $p_0(\text{down}) = 0.878$
 - $F_{\text{max}}(\text{down}) = 6.7$ floors (mean)
- For ascending ($d = \text{up}$):
 - $p_0(\text{up}) = 0.842$
 - $F_{\text{max}}(\text{up}) = 4.7$ floors (mean)

These probabilities are adjusted based on floor height and crowd density in the elevator waiting area, as participants are more likely to choose the elevator at higher floors or when queues are minimal. For simplicity, the adjusted probability is modeled as:

$$P_{\text{stairs}}^{\text{adj}}(f, d) = P_{\text{stairs}}(f, d) \cdot e^{-\alpha \cdot (f - F_{\text{max}}(d))}$$

where α is a decay constant determining how quickly the probability decreases with increasing floor numbers beyond $F_{\text{max}}(d)$. This reflects the decreasing willingness to use stairs as the number of floors increases.

c) Passenger Weight Distribution

Passenger weights affect elevator load and energy consumption. Anthropometric reference data from the *Vital and Health Statistics, Series 3, Number 46* [3] provides average weights by age and gender for the United States population between 2015 and 2018.

Passenger weights W are modeled using a normal distribution:

$$W \sim \mathcal{N}(\mu_w, \sigma_w^2)$$

where μ_w is the mean weight and σ_w is the standard deviation, calculated separately for males and females across different age groups. In the simulation, passengers are assigned weights based on these distributions, contributing to the dynamic load on the elevator.

d) Passenger Area Calculation

To estimate the area occupied by a standing passenger in the elevator, we use an elliptical model based on the passenger's weight and gender. This approach accounts for average body proportions, with separate coefficients for males and females. The area A of a passenger is computed as follows:

$$A = \pi \cdot a \cdot b$$

where a and b are the semi-major and semi-minor axes of the ellipse, respectively. These values depend on the passenger's weight W (in kg) and gender, defined by the following equations:

- For males:

$$a = 0.6 + 0.004 \cdot W, \quad b = 0.3 + 0.003 \cdot W$$

- For females:

$$a = 0.5 + 0.004 \cdot W, \quad b = 0.3 + 0.003 \cdot W$$

This formula allows the simulation to dynamically calculate each passenger's occupied area, ensuring accurate modeling of elevator space usage based on individual physical characteristics.

e) Boarding and Alighting Times

The time passengers take to board and alight the elevator influences overall system performance, affecting dwell times at floors and passenger waiting times. Boarding and alighting times are derived from the study by Seriani et al. [4], which, although focused on metro stations, provides relevant insights due to similar passenger behaviors.

Boarding times (T_b) and alighting times (T_a) are modeled using log-normal distributions, capturing the skewness observed in empirical data:

$$T_b \sim \text{Lognormal}(\mu_b, \sigma_b^2), \quad T_a \sim \text{Lognormal}(\mu_a, \sigma_a^2)$$

The parameters μ_b , σ_b , μ_a , and σ_a are estimated from the study's reported mean and variance of boarding and alighting times.

f) Commuting Activity Patterns

Due to the lack of directly applicable studies, commuting activity patterns are synthesized using predictive analytics based on datasets related to work commutes and recreational activities. The `commuting_activity_by_age.csv` file is created by analyzing statistical data on population movements throughout the day.

The probability $P_{\text{commute}}(t, a)$ of a passenger commuting at time t with age a is modeled using:

$$P_{\text{commute}}(t, a) = P_{\text{activity}}(t) \cdot P_{\text{age}}(a)$$

where:

- $P_{\text{activity}}(t)$ represents the overall activity level at time t .
- $P_{\text{age}}(a)$ is the probability distribution of passengers' ages.

These probabilities are used to adjust the passenger arrival rates and demographic distributions accordingly.

g) Integration into Simulation

By combining the factors above, the overall probability that a passenger will make an elevator call at time t is given by:

$$P_{\text{call}}(t) = \lambda(t) \cdot P_{\text{stairs}}^{\text{adj}}(f, d) \cdot P_{\text{commute}}(t, a)$$

To simulate a realistic and dynamic flow of passengers in the elevator system, each time step generates passengers based on the `accumulated_entity_probability`. The `PopulationGenerator` integrates this probability-based model to dynamically add entities to the queue, governed by the `max_human` variable, which sets a limit on the maximum number of entities to be generated across all `Population` algorithms. For example, if `max_human` is set to 1,000, entities will continue to be generated until this total count is reached. The simulation control algorithm subsequently leverages the queue to manage entities' boarding sequences and calculate their respective wait times, providing real-time insights into passenger flow. This probability accounts for the time-dependent arrival rates, the adjusted preference for using stairs or elevators, and the commuting activity patterns based on age. In the simulation, elevator calls are generated using a stochastic process where, at each time step, passengers are generated according to $P_{\text{call}}(t)$. Their weights, boarding times, and alighting times are sampled from the respective distributions.

h) Implementation in Code

In the simulation code, these external variables are implemented as follows:

i) Integration into Simulation

By combining the factors above, the overall probability that a passenger will make an elevator call at time t is given by:

$$P_{\text{call}}(t) = \lambda(t) \cdot P_{\text{stairs}}^{\text{adj}}(f, d) \cdot P_{\text{commute}}(t, a)$$

To simulate a realistic and dynamic flow of passengers in the elevator system, each time step generates passengers based on the `accumulated_entity_probability`. The `PopulationGenerator` integrates this probability-based model to dynamically add entities to the queue, governed by the `max_human` variable, which sets a limit on the maximum number of entities to be generated across all `Population` algorithms. For example, if `max_human` is set to 1,000, entities will continue to be generated until this total count is reached. The simulation control algorithm subsequently leverages the queue to manage entities' boarding sequences and calculate their respective wait times, providing real-time insights into passenger flow. This probability accounts for the time-dependent arrival rates, the adjusted preference for using stairs or elevators, and the commuting activity patterns based on age. In the simulation, elevator calls are generated using a stochastic process where, at each time step, passengers are generated according to $P_{\text{call}}(t)$. Their weights, boarding times, and alighting times are sampled from the respective distributions.

j) Implementation in Code

In the simulation code, these external variables are implemented as follows:

- **Passenger Arrival Generation:** To manage passenger arrivals more accurately, the `PopulationGenerator` utilizes an `accumulated_entity_probability` parameter, distributing generated entities across each simulation hour evenly. This distribution is subject to the constraint set by the `max_human` variable, which ensures that the total number of generated entities does not exceed the predefined maximum count. When the `total_time_passed` variable exceeds the `hour_length` value, the simulation time increments by one hour. The distribution is calculated using:

$$\text{Entity Probability} = \frac{\text{Average Entity per Hour} \cdot \Delta\text{Time}}{\text{Hour Length}}$$

- **Waiting Time Calculation:** A queue mechanism, `queue_entities: Vec<Vec<Box<dyn Boardable>>>` allows each generated entity to be added directly to a structured queue. This queue is then utilized by the control algorithm to assign each entity to an elevator, retaining a record that enables waiting time calculations for all entities. Each entity's wait time thus accumulates in the queue until it boards, allowing precise monitoring of individual waiting durations.
- **Stairs vs. Elevator Decision:** For each passenger, $P_{\text{stairs}}^{\text{adj}}(f, d)$ is computed to decide whether they will choose the stairs or the elevator. If the random draw falls below this probability, the passenger uses the stairs; otherwise, they call the elevator.
- **Passenger Attributes:** Passengers are assigned weights sampled from the normal distribution $W \sim \mathcal{N}(\mu_w, \sigma_w^2)$, and boarding and alighting times from the log-normal distributions for T_b and T_a .

k) Considerations and Assumptions

While the data and models provide a realistic basis for the simulation, certain assumptions are made due to limitations:

- The study by Kinsey et al. focused on evacuation scenarios, which may not perfectly represent daily elevator usage. However, similar behavioral tendencies are assumed to apply.
- Commuting activity patterns are synthesized due to the lack of specific datasets, potentially introducing uncertainties.
- The decay constant α in the adjusted stairs preference probability is calibrated based on reasonable assumptions to fit observed behaviors.

l) Conclusion

By integrating these external variables, the simulation creates a realistic and dynamic human traffic environment within the elevator system. This comprehensive modeling allows for a thorough evaluation of the control algorithms under test, ensuring that results are reflective of real-world scenarios.

3) Graphical User Interface (GUI)

The Graphical User Interface (GUI) is essential for user interaction with the elevator simulation, enabling parameter configuration, real-time performance monitoring, and outcome visualization. Implemented in Python using the customtkinter library for its modern aesthetics and enhanced functionality, the GUI communicates with the Rust-based simulation backend through FFI connections, ensuring efficient data handling and real-time updates.

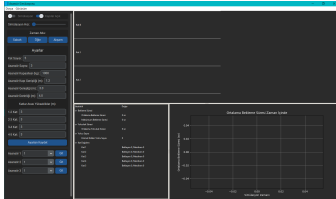


Fig. 1: Enter Caption

a) Interface Layout and Functionalities

The GUI is organized into key sections that facilitate an intuitive and efficient workflow:

- **Menu Bar:** Positioned at the top, the menu bar provides access to essential functions such as exiting the application and toggling the visibility of various GUI components. It includes two primary menus:
 - **File Menu:** Contains options like "Exit" to terminate the application.
 - **View Menu:** Offers checkboxes to show or hide the settings panel, elevator controls, statistics, and time skip sections, enhancing user customization.

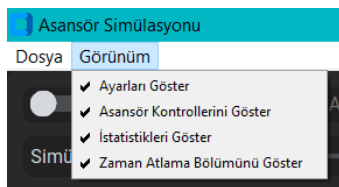


Fig. 2: The GUI Menu Bar with View Options

- **Control Panel:** Located on the left, the control panel allows users to manage simulation operations with features including:
 - **Simulation Switch:** Start or pause the simulation.
 - **Door Control Switch:** Open or close the elevator doors.
 - **Simulation Speed Slider:** Adjust the simulation speed dynamically.
 - **Time Skip Buttons:** Jump to specific times (e.g., morning, midday, evening) to observe elevator behavior under different scenarios.

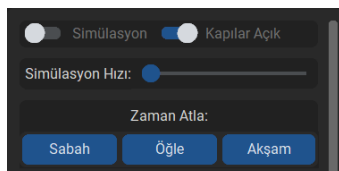


Fig. 3: The GUI Simulation Control Panel

- **Settings Panel:** This section allows users to configure various simulation parameters such as the number of floors, number of elevators, elevator capacity, door width, and floor heights. Users can input values directly or use sliders for precise adjustments. A "Save Settings" button applies the changes.

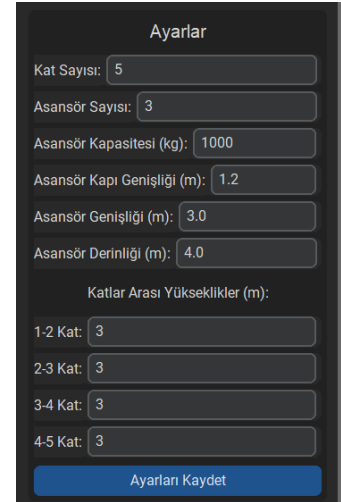


Fig. 4: The GUI Settings Panel

- **Manual Elevator Controls:** Situated below the settings panel, this area provides manual control over individual elevators, allowing users to select target floors for direct intervention and scenario testing.

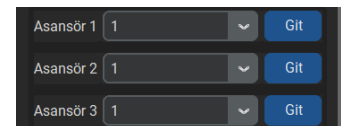


Fig. 5: The GUI Manual Elevator Controls

- **Visualization and Statistics Panel:** The central section includes a canvas displaying the building's floors and real-time elevator positions with color-coded operational status indicators. Adjacent to this, a statistics frame offers grouped data and dynamic visualizations, including a Treeview table for metrics like average and maximum wait times, journey times, and passenger counts, along with matplotlib plots for performance trend analysis.

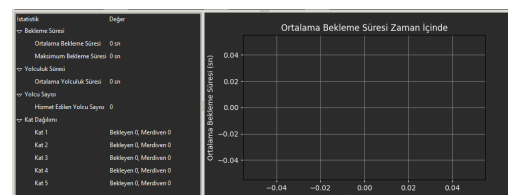


Fig. 6: The GUI Visualization and Statistics Panel

b) Technical Implementation

The GUI is developed using Python's `customtkinter` library, chosen for its modern aesthetics and enhanced widget capabilities over the standard `tkinter` framework. This library allows for an intuitive design, blending functionality with a clean, responsive interface that provides a streamlined user experience. To support real-time data visualization, `matplotlib` is integrated, enabling dynamic, high-quality plots that convey key metrics effectively.

- **Modular Design:** The GUI is structured around core modules, such as the Menu Bar, Control Panel, and Visualization-Statistics Panel, each handling specific tasks to promote an organized code structure and ease of maintenance. Key methods like `create_menu` and `create_control_frame` facilitate straightforward access to settings and control features, streamlining user interaction. The `create_stats_frame` method encapsulates the statistics display, presenting grouped data, trend analysis, and user-defined parameters in a structured, accessible format. This modular design enhances readability, future scalability and allows each component to be developed and updated independently.
- **Responsive Layout:** To ensure compatibility across various devices and screen sizes, the GUI incorporates `ttk.PanedWindow` and `customtkinter.Frame` widgets, which dynamically adapt the layout to maintain functionality and visual appeal. Horizontal and vertical panes are implemented to optimize space allocation, with the Control Panel and Visualization Panels proportionally resizing based on window dimensions. This responsiveness guarantees a consistent and user-friendly experience, whether accessed on a desktop or laptop, making the GUI versatile for different use cases and user environments.
- **Real-Time Communication via FFI:** Transitioning from socket connections, the GUI now uses Foreign Function Interface (FFI) to communicate directly with the Rust-based simulation backend, enhancing speed and reliability. Through the `pyo3` library, Rust functions are exposed as Python-callable methods, enabling seamless invocation of complex Rust functionalities directly from the Python GUI. For instance, Python can call `el.update_settings` to adjust configuration parameters or `el.run_simulation` to start and control the simulation process. This FFI-based setup fosters efficient, low-latency data exchange, reducing overhead compared to socket-based communication and enabling the GUI to reflect changes in real time with minimal delay.
- **Error Handling:** Comprehensive error management is integrated to enhance reliability and user experience. Invalid inputs are promptly flagged, and communication errors between the GUI and backend are managed gracefully. By using Python's `messagebox` widget, the GUI provides user-friendly prompts that guide users through corrections, such as saving configurations or adjusting settings before proceeding. This layered approach to error handling ensures that potential disruptions are minimized, supporting a smoother, more stable interaction.

- **Visualization Integration:** Real-time metrics are visualized within the GUI using `matplotlib` plots embedded via `FigureCanvasTkAgg`, allowing users to monitor trends and key performance indicators throughout the simulation. For instance, the `update_gui` method dynamically updates elevator positions, passenger counts, and waiting times, reflecting simulation states on the `Canvas` widget. These visualizations include customizable charts, with historical data plots showing trends over time for critical values like waiting times and journey durations. The ability to plot performance metrics in real time facilitates data-driven decision-making and deeper insight into simulation behavior.

c) Future Enhancements

Planned improvements are focused on expanding the GUI's functionality, enhancing visual engagement, and providing more advanced customization options:

- **Enhanced Visualization Tools:** Future updates aim to incorporate 3D modeling features, creating a more immersive and interactive simulation environment. These additions will enable users to view elevators and building layouts from multiple perspectives, enhancing both usability and analytical potential.
- **Motor Customization:** A proposed feature will allow users to directly modify the `motor_samples.csv` file within the GUI, simplifying motor parameter adjustments. This feature will enable users to experiment with different motor settings, analyze their effects on performance, and optimize configurations for specific scenarios without leaving the application.
- **Advanced Analytics and Reporting:** To support deeper insights, the GUI will include exportable reports, offering users a way to save detailed logs, performance metrics, and visualizations. These reports can be used for post-simulation analysis or shared with stakeholders, adding a layer of flexibility for professional or educational applications.

d) Conclusion

The meticulously designed GUI is pivotal to the elevator simulation, offering a cohesive interface for configuring simulation parameters, monitoring real-time performance, and visualizing outcomes. By leveraging Python's `customtkinter` for an intuitive design and `matplotlib` for dynamic visualization, the GUI provides a comprehensive user experience, seamlessly integrating with the Rust backend through efficient FFI communication. With a well-structured modular design, responsive layout, and advanced real-time data exchange capabilities, the GUI ensures a reliable and engaging platform for simulation interaction. Planned enhancements, including 3D visualization, motor customization, and advanced reporting, will continue to support the ongoing development and optimization of the simulation, enriching user engagement and functionality.

4) Additional Features

a) Boardable Trait

The `Boardable` Trait in our Rust-based elevator simulation allows flexible representation of any entity entering the elevator. Instead of being limited to human passengers, our system treats each entrant as an object, enabling diverse entities, such as animals or heavy loads, to be accommodated seamlessly. This approach provides a robust framework for versatile simulation scenarios.

Object Representation

Each entity is represented as an object with attributes such as weight, destination floor, and size, defining its interaction with the elevator. This abstraction enables the simulation to handle diverse load distributions, capturing variations in elevator occupancy and capacity.

Customization and Extensibility

A key advantage of the `Boardable` system is its extensibility. Users can create customized objects tailored to specific simulation needs, allowing scenarios like transporting large equipment or non-standard entities, thereby enhancing the simulation's applicability in both research and practical settings.

Implementation in Simulation Workflow

The `Boardable` Trait integrates into the simulation workflow as follows:

- 1) **Object Creation:** Users define new objects by specifying attributes relevant to the simulation.
- 2) **Assignment to Elevators:** These objects are assigned to elevators based on criteria such as load, proximity, and capacity.
- 3) **Simulation Execution:** The simulation accounts for the unique characteristics of each `Boardable` object, impacting travel time and load distribution.

Advantages of the Boardable Trait

The `Boardable` paradigm provides several key benefits:

- **Flexibility:** Accommodates scenarios beyond standard passenger transport.
- **Scalability:** New objects can be added seamlessly as simulation requirements evolve.
- **Realism:** Models non-human entities, mirroring complex real-world elevator usage.

b) Queue Struct

In our Rust-based elevator simulation, the `queue` struct, alongside the `PopulationGenerator`, manages entities waiting at each floor. This variable is defined as a vector of vectors, `queue_entities: Vec<Vec<Box<dyn Boardable>>>`, where each inner vector represents the queue of entities on a specific floor. Organizing entities in this manner enables efficient access and dynamic management, creating realistic passenger flow within the system.

Entity Management and Organization

The `queue` struct structures entities by floor. Each floor has its dedicated queue, where entities, such as passengers or cargo, wait for an elevator. Entities within each queue have attributes defined by the `Boardable` trait, including weight, destination, and wait time, allowing the simulation to accurately model demand across floors. The `PopulationGenerator` uses a parameter, `accumulated_entity_probability`, to evenly distribute entities generated per hour, calculated by:

$$\text{Entity Probability} = \frac{\text{Average Entity per Hour} \cdot \Delta\text{Time}}{\text{Hour Length}}$$

This ensures a smooth inflow of entities, adding each to the respective floor queue.

Handling Population and Dynamic Flow

The `queue` structure, combined with the `PopulationGenerator`, allows the simulation to manage population changes dynamically. New entities can be added, with wait times updated periodically, providing realistic, time-accurate behavior. The queue ensures each elevator has access to an up-to-date list of waiting entities, supporting efficient load and unload operations based on availability.

Advantages of the Queue Structure

The queue-based approach to managing entities offers several advantages:

- **Efficiency in Load Management:** Elevators access floor queues to board entities based on load and capacity.
- **Flexibility in Expansion:** Additional floors or specialized queues can be added easily without significant changes, allowing scalability.
- **Realistic Wait Time Tracking:** The `PopulationGenerator` and `accumulated_entity_probability` enable accurate tracking of entity wait times, adding depth to the simulation.

Future Enhancements

Planned improvements for the `queue` system aim to enhance functionality and simulation fidelity:

- **Priority-Based Queueing:** Introducing prioritization in the queue, allowing entities with urgent needs to board earlier.
- **Adaptive Queue Management:** Implementing adaptive algorithms to manage queues more effectively based on real-time elevator and floor load.
- **Integration with Analytics:** Using queue data for analytics, providing insights into peak times, high-demand floors, and bottlenecks.

5) Simulation Testing Methods

Our simulation undergoes rigorous testing through dedicated unit tests for each key component, including the motor, elevator, and elevator system. Each of these units has specialized tests designed to validate functionality and performance under various scenarios. Additionally, separate unit tests cover the code responsible for parsing configuration parameters, ensuring accurate setup for each simulation run.

These unit tests streamline the debugging process by allowing for targeted troubleshooting, where specific components can be tested independently to identify and address issues efficiently. By regularly running these tests, we ensure that each module behaves as expected, supporting the robustness and reliability of the overall simulation.

a) How to Run Simulation

The simulation is executed by running the compiled binary of the Rust project. To ensure optimal performance, particularly during the training of neural networks and the operation of PID controllers, it is highly recommended to build the project in release mode. This can be achieved using the following command:

```
cargo build --release
```

While the standard build command (`cargo build`) compiles the project quickly, the resulting executable operates significantly slower, which can hinder the effectiveness of training processes and control algorithms. In contrast, the release build undergoes optimizations that enhance runtime performance, making it suitable for intensive tasks.

After building, the generated executable (named `main.exe` on Windows or simply `main` on Linux) can be executed with specific command-line arguments to perform various operations:

- `--train`: Initiates the training process for the neural networks.
- `--algorithm=simple` or `--algorithm=neural`: Selects the desired control algorithm to be used in the simulation.

For example, to start the training of neural networks, the following command can be used:

```
main.exe --train
```

Alternatively, to run the simulation using the Simple Elevator Control Algorithm, execute:

```
main.exe --algorithm=simple
```

Furthermore, the simulation can be run directly in release mode with specific arguments without manually building the release binary by utilizing the `cargo run` command as shown below:

```
cargo run --release -- --train
```

This command compiles and runs the project in release mode in a single step. It is essential to use the release build configuration when performing training and deploying control algorithms to ensure that the simulation operates efficiently

and achieves accurate results. Failure to do so may result in suboptimal performance and unreliable outcomes.

Additionally, the `main.exe` executable supports both training and algorithm selection through command-line arguments, providing flexibility in managing different simulation scenarios. By adhering to the release build process and utilizing the appropriate command-line options, users can effectively leverage the full capabilities of the simulation framework, ensuring both efficiency and reliability in elevator control optimization tasks.

Originally, the plan was to have all functionalities accessible via a graphical user interface. However, due to time constraints, the GUI integration has been postponed. The core interface definitions and FFI (Foreign Function Interface) connections from the Rust side are already implemented, and they only need to be imported into the GUI. This will be set up when time allows, ensuring a complete and user-friendly interface for the simulation.

B. Algorithms

The core of the study involves the implementation and comparison of both traditional and artificial intelligence-based elevator control algorithms. Each algorithm is described in detail below.

a) Neuroevolution-Based Control Algorithm

To optimize elevator dispatching and minimize passenger wait times, we implemented a Neuroevolution-based control algorithm. This algorithm leverages a neural network trained through evolutionary strategies to determine the optimal floor assignments for each elevator in real-time.

Neural Network Architecture

The control algorithm's neural network is designed with three hidden layers, providing sufficient depth to capture complex relationships between the system's state and the optimal elevator actions. The architecture is as follows:

- **Input Layer:** The input vector represents the current state of the elevator system, including the number of passengers waiting on each floor and the status of each elevator. To maintain consistency, this vector is fixed in length, regardless of the number of passengers.
- **Hidden Layers:** Three fully connected hidden layers, each utilizing the Rectified Linear Unit (ReLU) activation function, enable the network to learn non-linear mappings from inputs to outputs.
- **Output Layer:** The output layer produces a vector where each element corresponds to the target floor for an elevator. A softmax activation function is applied to convert the outputs into probability distributions, facilitating probabilistic decision-making.

Training Methodology

The neural network's weights and biases are initially assigned random values to provide diverse starting points for the optimization process. The training procedure employs an evolutionary strategy, which iteratively improves network performance through selection and mutation. The training process comprises the following steps:

- 1) **Population Initialization:** A population of neural networks is created, each with randomly initialized weights and biases, ensuring a diverse set of candidate solutions.
- 2) **Evaluation:** Each network in the population is evaluated by running multiple simulation scenarios where passengers are randomly distributed across floors. The performance of each network is assessed using a reward function that primarily considers the total passenger wait time.
- 3) **Selection:** Networks are ranked based on their performance scores. The top-performing networks are selected to serve as parents for the next generation.
- 4) **Mutation:** Selected networks undergo mutations, where their weights and biases are slightly altered based on predefined mutation probabilities and magnitudes. This introduces variability and allows exploration of new regions in the parameter space.
- 5) **Generation Advancement:** The mutated networks replace the lower-performing networks in the population, and the process repeats for a specified number of generations or until convergence is achieved.

To ensure robustness and prevent overfitting to specific passenger distributions, each network is evaluated across multiple runs with varying passenger assignments. The average performance across these runs provides a reliable metric for selection, enhancing the generalizability of the trained networks.

Reward Function

The reward function is designed to incentivize the reduction of passenger wait times while maintaining energy efficiency. It is defined as:

$$\text{Reward} = -\text{Total Wait Time} - \text{Energy Consumption}$$

In this study, the focus is primarily on minimizing wait times, as the energy consumption of the motors is relatively low. Therefore, the reward function primarily reflects the negative total wait time, with energy consumption serving as a secondary consideration for future extensions.

Evolutionary Optimization

The training process employs a Genetic Algorithm (GA) framework to optimize the neural network's weights and biases. Key parameters include:

- **Mutation Probability (P_m):** Determines the likelihood of each weight or bias undergoing mutation during the training process.
- **Mutation Magnitude (M):** Controls the extent of the perturbations introduced during mutation, balancing exploration and exploitation.
- **Evolutionary Pressure:** Defines the proportion of the population retained across generations, influencing the selection intensity.

These parameters are fine-tuned based on empirical observations to ensure efficient convergence towards optimal network configurations.

Integration with Elevator Control System

Once trained, the neural network serves as the core decision-making component within the elevator control system. At each simulation timestep, the current system state vector is fed into the network, which outputs target floor assignments for each elevator. The control controller then directs the elevators accordingly, dynamically adapting to real-time passenger demand patterns to optimize overall system performance.

Handling Multiple Elevators and Passengers

Given the stochastic nature of passenger arrivals and destination assignments, the network must generalize across a wide range of scenarios. To address potential biases where elevators might be assigned to only a subset of floors, each training iteration involves multiple passenger distributions. The performance metrics are averaged across these distributions, ensuring that the network learns to handle diverse and unpredictable passenger flows effectively.

Performance Optimization

The training process parameters, such as mutation probability and mutation magnitude, are tuned to balance exploration and exploitation. An aggressive mutation rate accelerates the exploration of the parameter space but may hinder convergence, whereas a conservative rate promotes stable optimization but at the expense of exploration. Empirical tuning based on simulation performance guides the selection of these hyperparameters to achieve optimal training outcomes.

Advantages and Limitations

The Neuroevolution-based approach offers significant advantages in adaptability and performance optimization, particularly in dynamic and high-traffic environments. By learning from diverse scenarios, the neural network can generalize its decision-making process, effectively reducing wait times. However, the evolutionary training approach is computationally intensive and may require substantial simulation runs to achieve convergence, especially as the complexity of the elevator system increases.

Future Enhancements

Future work includes integrating multi-objective optimization to concurrently minimize wait times and energy consumption. Additionally, exploring more sophisticated evolutionary strategies or hybrid approaches that combine evolutionary strategies with gradient-based methods could potentially enhance training efficiency and network performance.

b) Simple Elevator Control Algorithm

In addition to the Neuroevolution-based approach, we implemented a *Simple Elevator Control Algorithm* as a baseline for comparison. This algorithm employs a deterministic strategy to manage elevator dispatching, focusing primarily on minimizing passenger wait times without leveraging machine learning techniques. Its simplicity allows for straightforward implementation and provides a clear benchmark against which the performance of AI-driven methods can be assessed.

Algorithm Overview

The Simple Elevator Control Algorithm is designed to efficiently manage two elevators within a building, assigning target floors based on a predefined strategy. The core idea is to designate one elevator for upward movement and the other for downward movement, systematically cycling through target floors to ensure balanced service across all floors. This deterministic approach ensures predictable behavior, making it suitable for environments with consistent and manageable passenger traffic patterns.

Controller Structure

The controller maintains the state of each elevator, including their current target floors and operational status. Key components of the controller include:

- **Elevator Indices:** Two indices, `up_elevator_idx` and `down_elevator_idx`, track which elevator is designated for upward and downward movements, respectively.
- **Target Floors:** `up_current_target` represents the current target floor for the upward-moving elevator. The downward-moving elevator's target is dynamically calculated based on this value to ensure complementary operation.
- **Initialization Flag:** A boolean flag, `initialized`, ensures that the elevators are properly initialized before commencing regular operations.
- **Maximum Floor:** `max_floor` stores the highest floor number in the building, facilitating boundary checks and target floor assignments.

Initialization Process

Upon the first update cycle, the controller initializes the elevators by setting their initial target floors. The process involves:

- 1) **Setting Maximum Floor:** The controller retrieves the total number of floors from the elevator configuration and calculates the `max_floor` as one less than the total number of floors (assuming zero-based indexing).
- 2) **Assigning Initial Targets:** The upward-moving elevator is assigned the first floor (ground floor) as its target, while the downward-moving elevator is assigned the top floor (`max_floor`).
- 3) **Flagging Initialization Completion:** Once the elevators reach their initial target floors, the `initialized` flag is set to `true`, indicating that the system is ready to commence regular operations.

Operational Logic

The operational cycle of the Simple Elevator Control Algorithm consists of the following steps:

- 1) **Target Floor Assignment:** The upward-moving elevator is incrementally assigned target floors in ascending order, while the downward-moving elevator mirrors this assignment by targeting floors in descending order. This systematic approach ensures that each elevator services a distinct set of floors, reducing potential conflicts and optimizing coverage.

- 2) **Elevator Swapping:** Once the upward-moving elevator reaches the top floor (`max_floor`), the roles of the elevators are swapped. The previously upward-moving elevator becomes the new downward-moving elevator, and vice versa. This swapping mechanism ensures continuous and balanced service across all floors.
- 3) **Passenger Loading and Unloading:** At each target floor, the algorithm manages the loading and unloading of passengers. The controller checks if the elevator can board additional passengers based on capacity constraints and directs eligible passengers to board. Passengers with destinations aligned with the elevator's current direction are prioritized to maintain efficiency.
- 4) **Cycle Continuation:** The controller continuously updates target floors and manages elevator assignments in a cyclical manner, ensuring that both elevators operate harmoniously and provide consistent service throughout the simulation.

Advantages and Limitations

Advantages:

- **Simplicity:** The algorithm's straightforward logic facilitates easy implementation and debugging.
- **Predictability:** Deterministic behavior ensures consistent performance, making it suitable for environments with stable and predictable passenger flows.
- **Low Computational Overhead:** Unlike AI-based methods, the Simple Elevator Control Algorithm requires minimal computational resources, allowing for efficient real-time operation.

Limitations:

- **Lack of Adaptability:** The algorithm does not adapt to dynamic changes in passenger demand patterns, potentially leading to inefficiencies in high-traffic or unpredictable scenarios.
- **Fixed Strategy:** The predetermined assignment of elevators to specific directions may result in suboptimal performance when passenger distributions do not align with the fixed strategy.
- **Scalability Constraints:** While effective for systems with a limited number of elevators and floors, the algorithm may struggle to maintain efficiency in larger or more complex buildings.

Conclusion

The Simple Elevator Control Algorithm offers a foundational strategy for managing elevator dispatching in multi-elevator systems. Its simplicity and predictability make it a valuable benchmark for more advanced, adaptive control methods. While effective in controlled environments with stable passenger flows, its limitations underscore the necessity for adaptive algorithms like Neuroevolution-based controllers to handle dynamic and complex operational scenarios efficiently.

C. Implementation Details

The algorithms are implemented within a Rust-based simulation framework that models the physical dynamics of elevators and motor components. Key implementation aspects include:

- **PID Controllers:** Both elevator and motor dynamics are regulated using PID controllers to ensure precise speed and acceleration adjustments.
- **Energy Consumption Tracking:** The simulation tracks the total energy consumed by each elevator, providing a basis for evaluating the efficiency of different algorithms.
- **Configuration Files:** YAML configuration files define parameters for elevators, motors, and control algorithms, enabling easy customization and scalability of the simulation.

D. Algorithm Evaluation Metrics

To assess the performance of each algorithm, the following metrics are considered. These metrics provide a comprehensive understanding of how effectively each control strategy manages elevator operations, optimizes energy usage, and enhances passenger experience.

- **Average Waiting Time:** This metric measures the average duration that passengers wait for an elevator from the moment they request service until the elevator arrives at their floor. It is a critical indicator of passenger satisfaction and system responsiveness. Lower average waiting times signify more efficient elevator dispatching and better handling of passenger flows, especially during peak hours. This metric is calculated by summing all individual passenger wait times and dividing by the total number of passengers served during the simulation period.
- **Energy Consumption:** Energy consumption quantifies the total electrical energy utilized by the elevator system over a specified timeframe. It encompasses the energy used for moving the elevators, operating the motors, and powering auxiliary systems such as lighting and control units. Minimizing energy consumption is essential for reducing operational costs and enhancing the sustainability of the building. This metric is measured in kilowatt-hours (kWh) and is calculated by integrating the power usage over the duration of the simulation.
- **System Throughput:** System throughput refers to the number of passengers transported by the elevator system within a given period. It reflects the capacity and efficiency of the control algorithm in handling passenger demand. Higher throughput indicates that the system can accommodate more passengers without significant delays, making it particularly important in high-traffic environments. Throughput is typically measured by counting the total number of passengers successfully transported from their origin to their destination floors during the simulation.
- **Efficiency:** Efficiency is a composite metric that evaluates the overall performance of the elevator control algorithm by considering multiple factors such as energy consumption, wait times, and throughput. It provides a holistic view of how well the algorithm balances operational costs with service quality. Higher efficiency indicates that the system effectively minimizes energy usage and passenger wait times while maximizing throughput. This metric can be calculated using a weighted formula that assigns relative importance to each individual metric based on

the specific goals of the elevator system.

a) Metric Calculation Details

- **Average Waiting Time** is computed as:

$$\text{Average Waiting Time} = \frac{\sum_{i=1}^N W_i}{N}$$

where W_i is the wait time for the i -th passenger and N is the total number of passengers.

- **Energy Consumption** is calculated by integrating the power usage over time:

$$E_{\text{total}} = \int_{t_0}^{t_f} P(t) dt$$

where $P(t)$ is the instantaneous power consumption at time t , and t_0 to t_f represents the simulation period.

- **System Throughput** is determined by counting the number of passengers served:

$$\text{Throughput} = \sum_{i=1}^N 1_{\text{served}}(i)$$

where $1_{\text{served}}(i)$ is an indicator function that equals 1 if the i -th passenger is successfully transported, and 0 otherwise.

- **Response Time** is measured as:

$$\text{Average Response Time} = \frac{\sum_{i=1}^N R_i}{N}$$

where R_i is the response time for the i -th passenger.

- **Efficiency** can be modeled using a weighted sum of normalized metrics:

$$\begin{aligned} \text{Efficiency} = & \alpha \left(1 - \frac{\text{Average Waiting Time}}{\text{Max Wait Time}} \right) \\ & + \beta \left(1 - \frac{\text{Energy Consumption}}{\text{Max Energy Consumption}} \right) \\ & + \gamma \left(\frac{\text{System Throughput}}{\text{Max Throughput}} \right) \end{aligned}$$

where α , β , and γ are weights assigned to each component based on their relative importance.

III. CONCLUSION

In this study, we developed and implemented two distinct elevator control algorithms—a Neuroevolution-based approach utilizing a neural network with three hidden layers and a Simple Elevator Control Algorithm based on deterministic dispatching strategies. Both algorithms were integrated into a comprehensive simulation environment designed to model realistic elevator dynamics and passenger behaviors.

Due to the extensive computational resources and time required for training the neural network, the training process remains incomplete at this stage. As a result, a thorough evaluation of the Neuroevolution-based algorithm's performance is pending. We encourage interested readers and collaborators to participate in the training process or await our forthcoming results, which will include a detailed comparative analysis of the implemented algorithms based on key performance metrics.

such as average waiting time, energy consumption, and system throughput.

Future work will focus on completing the training of the neural network, refining the control algorithms, and exploring additional AI-driven strategies to further enhance elevator system efficiency. By leveraging adaptive learning methods, we aim to develop intelligent vertical transportation systems capable of dynamically responding to fluctuating passenger demands and optimizing operational performance in real-world high-rise buildings.

Additionally, we plan to investigate the integration of multi-objective optimization techniques to simultaneously minimize wait times and energy consumption, thereby achieving a balanced improvement in both passenger experience and operational costs. The ongoing development of our simulation framework will also accommodate more complex scenarios, including varying passenger distributions and multi-elevator coordination, to better reflect the challenges faced in modern urban environments.

In summary, while the current phase of our research lays a solid foundation for intelligent elevator control systems, the completion of the training process and subsequent evaluations will provide deeper insights and validate the effectiveness of AI-based methodologies in optimizing elevator operations.

IV. ACKNOWLEDGEMENT

This project could not have been completed without the incredible support, humor, and culinary contributions of many. First and foremost, we extend our deepest thanks to Tuna's mom, Hilal Abila, for her endless supply of delicious köfte, fueling both our stomachs and our souls. A big thank you to Mehmet Ali, who kept our life functions going with his steady provision of çiğköfte and Red Bull, our indispensable survival essentials. And also, special thanks to Elif Duru, whose stunningly designed flowcharts are nothing short of artwork (we're convinced they belong in a museum, which is why we didn't include them in our paper—besides, Tuna didn't actually write the code for those algorithms—but they certainly deserve a showcase of their own!).

Our heartfelt gratitude goes to Richard M. Stallman, for devising the GPL-v3 license and helping us create in an environment of freedom and security. We'll be forever grateful to him for protecting our right to share and modify this code without fear of corporate overlords taking over.

And in a bizarre twist of inspiration, we'd also like to thank Ted Kaczynski—not for his "impactful" manifestos but for his *lack* of success, which has allowed us to keep typing away at keyboards rather than running from the woods. (Fun fact: there may or may not be a sneaky Easter egg dedicated to Ted somewhere in this code... but good luck finding it!)

A very special acknowledgment goes to the developers of ChatGPT, who have created this miraculous Large Language Model that guided us through writing both the paper and the code. Without ChatGPT-4o's assistance in formulating ideas, offering solutions, and handling the occasional writer's block, this project would have been significantly more challenging. It's worth noting that pieces of this paper—including

this acknowledgment section—are inspired by insights and suggestions from ChatGPT-4o, and LLM models have been invaluable in our software development journey.

Also we would appreciate our faithful coffee machine which is a beacon through the final sleepless nights, and, of course, to Frank Zappa, the legendary musician who kept our ears entertained through every line of code across three unforgettable, sleepless nights. Hats off to the sun for rising every morning, though we usually missed it (friends assure us it's a lovely sight), and to the delivery drivers who brought food to our doorstep with heroic dedication. Cheers as well to the mystery pub bartender who always had a cold drink ready to lift our spirits when we needed a break (actually, it was more about our friends' girlfriend troubles, but whatever...).

We can't forget Yüşa, whose legendary dreadlocks we miss dearly, and whose mere existence kept us smiling. Immense thanks to Cem Abi, who, in a single legendary act of kindness, came over one day and washed every dish we'd stacked up for about a month. And finally, to the secret legend, Toygar—despite being in Germany, your fascist dark memes on WhatsApp kept us laughing. You're missed, big man, and it's time for you to come home!

And the most important, endless appreciation to our grand and glorious benefactor, the robot overlords of NFR Products, for funding all our extravagant coffee purchases and more.

To all, you've made this journey "müthişş and iinnanılmaz" (as Tuna would have said). Thank you!

Finally, our sincere thanks to all the authors cited in our bibliography. Your work not only informed but also inspired this paper. We deeply appreciate the contributions of every researcher who paved the way for our understanding and growth.

I feel like this section could go on forever—and I say this as Yiğit, the one mainly handling the writing of this paper—but there's one more thank-you I must share, especially as we've ventured a bit beyond the usual scope here. I want to express my deep gratitude to my invaluable friend and collaborator Tuna, who not only made this process infinitely more enjoyable and insightful but also pulled countless all-nighters to help us shine in this project. Sharing some of the most important times in my life with you and witnessing your brilliant software expertise applied in this work has truly broadened my own perspective. Thank you for everything, my dear friend, one of the most cherished people in my life.

REFERENCES

- [1] I. M. Municipality, "Hourly public transport data set," 2024, accessed: 2024-04-27. [Online]. Available: <https://data.ibb.gov.tr/dataset/hourly-public-transport-data-set/resource/b3f67274-4ebc-4453-8f6b-1dccc1b1bf008/view/2f644c5a-ab10-4ae5-9136-c566ede1d67f>
- [2] M. Kinsey, E. Galea, and P. Lawrence, "Stairs or lifts? - a study of human factors associated with lift/elevator usage during evacuations using an online survey," in *Springer eBooks*, 2011, pp. 627–636.
- [3] Vital and N. . Health Statistics, Series 3, "Anthropometric reference data for children and adults: United states, 2015–2018," 2018.
- [4] S. Seriani, R. Fernandez, N. Luangboriboon, and T. Fujiyama, "Exploring the effect of boarding and alighting ratio on passengers' behaviour at metro stations by laboratory experiments," *Journal of Advanced Transportation*, vol. 2019, pp. 1–12, 2019.